

Geometrical transfer_



Contents

1 0. 오리엔테이션

1.1 본 자료의 구성

1.2 기하학적 변환이란

1.3 변환의 계층 — 무엇이 보존되는가

1.4 좌표 매핑과 보간

2 1. 이동 변환 (Translation)

2.1 이동 변환의 수식과 2×3 어파인 행렬

2.2 cv2.warpAffine() 함수

2.3 실습 코드 — 이동 변환

2.4 코드 들여다보기 — 사용한 함수·클래스 (1)

2.5 실행 결과 — 이동 변환

2.6 보충 — 강의 슬라이드의 이동 변환

2.7 실습 코드 — 이동 변환 (.py 버전)

2.8 실행 결과 (.py 버전)

3 2. 전단 변환 (Shear)

3.1 전단 변환의 수식

3.2 실습 코드 — 전단 변환

3.3 코드 들여다보기 — 사용한 함수·클래스 (2)

3.4 실행 결과 — 전단 변환

4 3. 영상의 확대와 축소 (Scaling)

4.1 크기 변환의 수식

4.2 cv2.resize() 함수

4.3 보간법(interpolation) 비교

4.4 실습 코드 — 보간법 4종 비교

4.5 코드 들여다보기 — 사용한 함수·클래스 (3)

4.6 원본 영상과 비교 구간

4.7 실행 결과 — NEAREST vs LINEAR

4.8 실행 결과 — CUBIC vs LANCZOS4

4.9 대칭 변환 — cv2.flip()

4.10 영상 피라미드 — cv2.pyrDown()

5 4. 회전 (Rotation)

5.1 회전 변환의 수식

5.2 실습 코드 — 원점 기준 회전(20°)

5.3 코드 들여다보기 — 사용한 함수·클래스 (4)

5.4 실행 결과 — 원점 기준 회전의 한계

5.5 cv2.getRotationMatrix2D() 함수

5.6 실습 코드 — 중앙 기준 회전 + 축소

5.7 코드 들여다보기 — 사용한 함수·클래스 (5)

5.8 실행 결과 — 중앙 기준 회전

5.9 직접 구현한다면 — 이동·회전·역이동

6 5. 어파인 변환과 투시 변환

6.1 두 변환의 비교

6.2 기하학적 차이

6.3 cv2.getAffineTransform() 함수

6.4 cv2.getPerspectiveTransform() 함수

6.5 cv2.warpAffine() 다시 보기

6.6 cv2.warpPerspective() 함수

6.7 어파인 vs 투시 정리

7 6. 응용 — 명함 펴기와 문서 스캐너

7.1 실습 코드 — 명함 펴기

7.2 코드 들여다보기 — 사용한 함수·클래스 (6)

7.3 실행 결과 — 명함 펴기

7.4 문서 스캐너 — 아이디어

7.5 실행 화면 (1) — 초기 사각형

7.6 실행 화면 (2) — 마우스로 모서리 맞추기

7.7 실행 화면 (3) — 변환 결과

7.8 실습 코드 (1) — ROI 그리기 함수

7.9 코드 들여다보기 — 사용한 함수·클래스 (7)

7.10 실습 코드 (2) — 마우스 콜백

7.11 코드 들여다보기 — 사용한 함수·클래스 (8)

7.12 실습 코드 (3) — 메인 루프

7.13 코드 들여다보기 — 사용한 함수·클래스 (9)

8 7. 정리

8.1 변환별 함수 정리표

8.2 자주 겪는 함정

8.3 학습 포인트

Geometrical transfer__

Exported from Confluence · <https://pinkwink.atlassian.net>

0. 오리엔테이션

본 자료의 구성

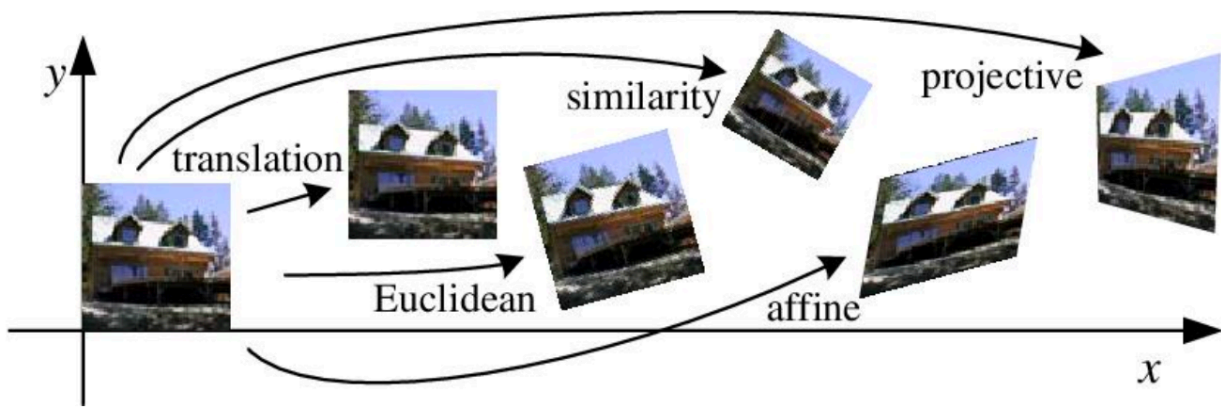
영상의 기하학적 변환 — 학습 흐름



학습 순서

- 기하학적 변환의 정의와 계층(이동 → 유클리드 → 닻음 → 어파인 → 투시)에서 출발
- 이동·전단 → 크기 → 회전 → 어파인·투시 순으로 자유도를 하나씩 늘려 감
- 각 변환마다 수식·행렬 → 함수 시그니처 → 실습 코드 → 실행 결과 4단계 반복
- 마무리는 투시 변환 응용 — 명함 퍼기, 마우스로 모서리를 잡는 문서 스캐너
- 실습 영상: tekapo.bmp, rose.bmp, pinkwink_namecard.png, scanned.jpg

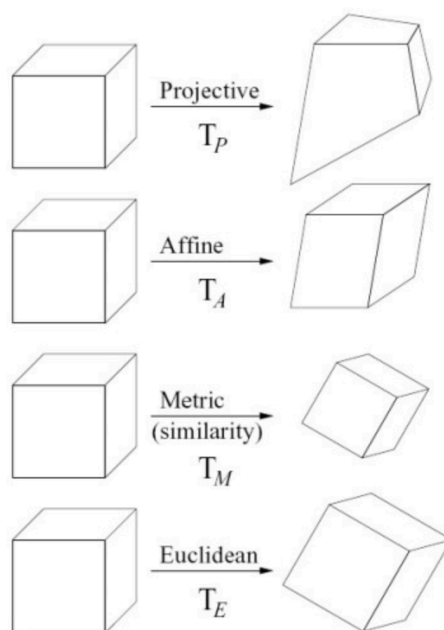
기하학적 변환이란



핵심 개념

- 기하학적 변환: 화소값이 아니라 화소의 위치(좌표)를 바꾸는 연산
- 밝기·필터링이 $\text{dst}(x, y) = f(\text{src}(x, y))$ 라면, 기하학적 변환은 $\text{dst}(x', y') = \text{src}(x, y)$
- 제약이 적어질수록 상위 변환 — translation $\subset$ Euclidean $\subset$ similarity $\subset$ affine $\subset$ projective
- 영상 정합, 파노라마, 문서 스캔, 데이터 증강(augmentation) 등에 폭넓게 사용

변환의 계층 — 무엇이 보존되는가



변환	자유도 (DOF)	보존되는 성질	OpenCV 함수
이동(translation)	2	방향·길이·각도·평행	<code>warpAffine</code> (2×3 직접 작성)

유클리드 (Euclidean)	3	길이·각도·평행	<code>getRotationMatrix2D</code> + <code>warpAffine</code>
닮음(similarity)	4	각도·평행(길이 비율만 유지)	<code>getRotationMatrix2D</code> (scale 사용)
어파인(affine)	6	평행성(직선은 직선으로)	<code>getAffineTransform</code> + <code>warpAffine</code>
투시(projective)	8	직선성만 유지(평행 깨짐)	<code>getPerspectiveTransform</code> + <code>warpPerspective</code>

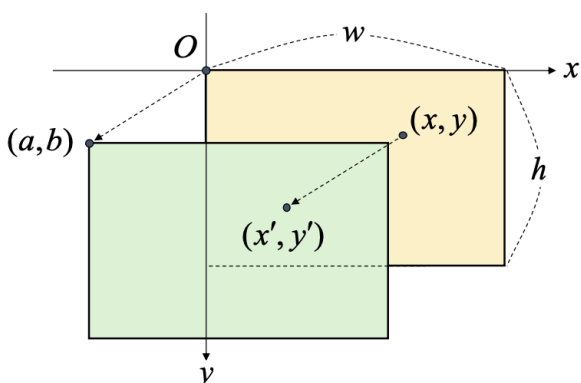
좌표 매핑과 보간

알아 둘 점

- 변환 후 좌표는 대부분 정수가 아닌 실수가 됨 → 주변 화소로부터 값을 추정해야 함
- 이 추정 과정이 **보간(interpolation)** — `flags` · `interpolation` 인자로 지정
- OpenCV는 실제로 **출력 좌표에서 입력 좌표를 역으로 찾는(inverse mapping)** 방식으로 계산 → 빈 구멍(hole) 없이 채워짐
- 출력 크기(`dsize`)를 `(0, 0)`으로 두면 입력과 같은 크기 → **변환으로 밀려난 부분은 잘림**
- 잘린 영역은 `borderMode` · `borderValue`에 따라 채워짐(기본은 검은색 0)

1. 이동 변환 (Translation)

이동 변환의 수식과 2x3 어파인 행렬



$$\begin{cases} x' = x + a \\ y' = y + b \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a \\ b \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2x3 어파인 변환 행렬

핵심 개념

- $x' = x + a$, $y' = y + b$ — x축으로 a, y축으로 b만큼 평행 이동
- 덧셈 형태를 행렬 곱 하나로 표현하기 위해 좌표에 1을 붙인 동차좌표(homogeneous) 사용
- 결과가 $\begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix}$ 형태의 2×3 어파인 변환 행렬
- OpenCV의 모든 어파인 계열 변환은 이 2×3 행렬 하나로 표현됨

cv2.warpAffine() 함수

```
cv2.warpAffine(src, M, dsize, dst=None, flags=None,
               borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

인자 정리

- M : 2×3 어파인 변환 행렬. 반드시 실수형(float32)
- dsize : 결과 영상 크기 (w, h) 튜플. (0, 0) 이면 입력과 같은 크기
- flags : 보간법. 기본값 cv2.INTER_LINEAR
- borderMode : 가장자리 처리 방식. 기본값 cv2.BORDER_CONSTANT
- borderValue : BORDER_CONSTANT일 때 채울 값. 기본 0(검은색)

실습 코드 — 이동 변환

```
import sys
import numpy as np
import cv2
```

✓ 0.1s

Python

```
src = cv2.imread('./data/tekapo.bmp')

aff = np.array([[1,0,200],
               [0, 1, 100]], dtype=np.float32)

dst = cv2.warpAffine(src, aff, (0,0))

cv2.imshow('src', src)
cv2.imshow('dst', dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

6.9s

Python

python

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread('./data/tekapo.bmp')
6
7  aff = np.array([[1, 0, 200],
8                 [0, 1, 100]], dtype=np.float32)    # x로 200, y로 100 이동
9
10 dst = cv2.warpAffine(src, aff, (0, 0))           # 출력 크기 = 입력과 동일
11
12 cv2.imshow('src', src)
13 cv2.imshow('dst', dst)
14 cv2.waitKey()
15 cv2.destroyAllWindows()
```

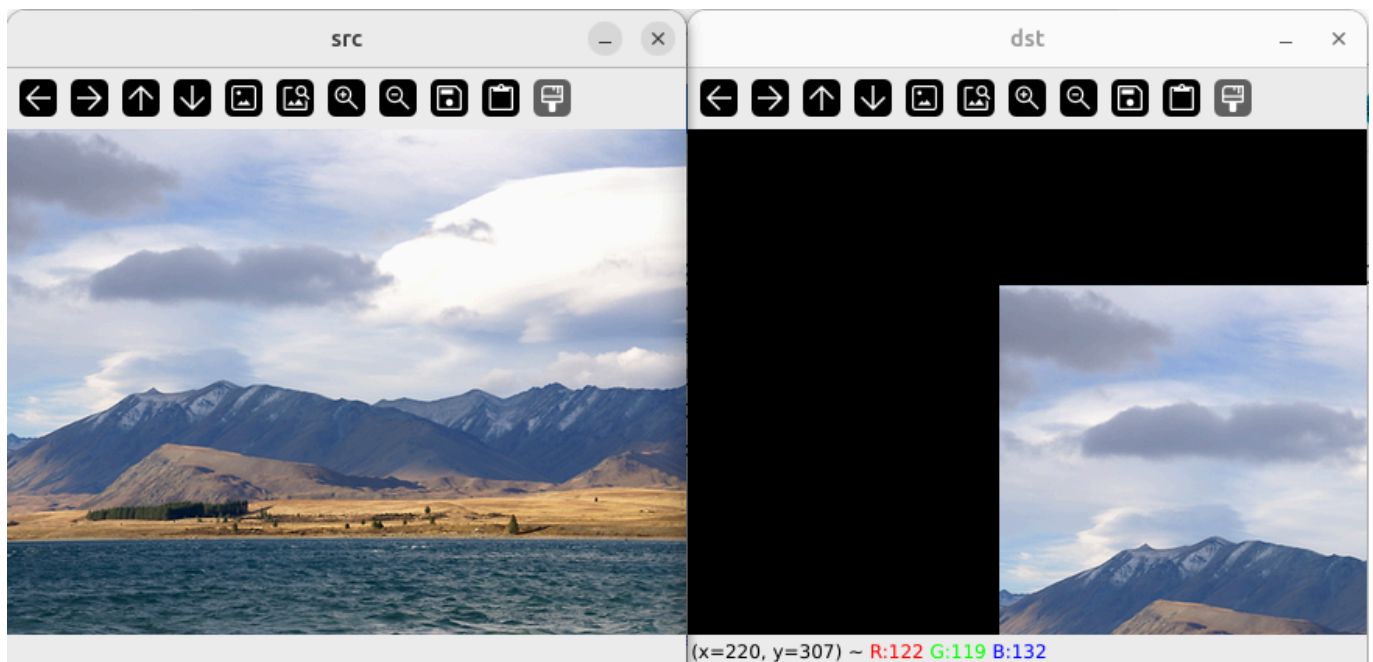
코드 들여다보기 — 사용한 함수·클래스 (1)

이름	종류	설명
sys	모듈	표준 라이브러리. 로드 실패 시 <code>sys.exit()</code> 에 사용
numpy (np)	모듈	변환 행렬을 만드는 행렬 연산 라이브러리
cv2	모듈	OpenCV 파이썬 바인딩
cv2.imread()	함수	영상 파일을 ndarray로 읽음

<code>np.array(..., dtype=np.float32)</code>	함수	2×3 이동 행렬 생성. float32 지정이 필수
<code>cv2.warpAffine()</code>	함수	2×3 행렬로 어파인 변환 수행
<code>(0, 0)</code>	인자	<code>dsize</code> — 입력과 같은 크기로 출력
<code>cv2.imshow() / waitKey() / destroyAllWindows()</code>	함수	결과 표시, 키 대기, 창 정리

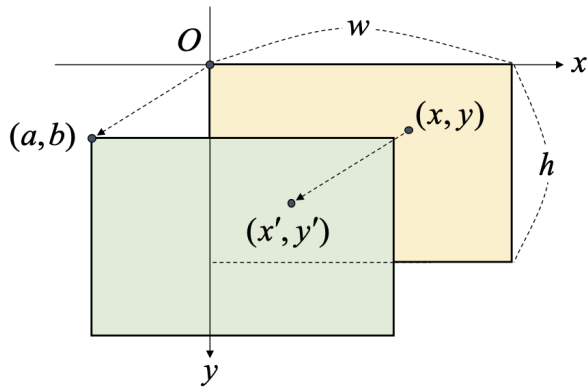
행렬을 `dtype` 없이 만들면 정수형(int64)이 되어 `warpAffine` 에서 오류가 남. 반드시 `np.float32` 로 지정할 것.

실행 결과 — 이동 변환



- 영상이 오른쪽 200, 아래로 100픽셀 이동
- 비워진 좌·상단은 `BORDER_CONSTANT` 기본값에 따라 **검은색**으로 채워짐
- 출력 크기를 입력과 같게 뒀으므로 밀려난 우·하단은 **잘려 사라짐**

보충 — 강의 슬라이드의 이동 변환



$$\begin{cases} x' = x + a \\ y' = y + b \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} a \\ b \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & a \\ 0 & 1 & b \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

↑
2x3 어파인 변환 행렬

```
cv2.warpAffine(src, M, dsize, dst=None, flags=None,
               borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

- 같은 내용을 다른 강의 자료로 한 번 더 정리한 슬라이드
- 이동 변환은 어파인 변환의 가장 단순한 형태임을 확인

실습 코드 — 이동 변환 (.py 버전)

```

src = cv2.imread("./data/tekapo.bmp")

if src is None:
    print("Image load failed!")
    sys.exit()

aff = np.array([[1, 0, 200],
                [0, 1, 100]], dtype=np.float32)

dst = cv2.warpAffine(src, aff, (0, 0))

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()

cv2.destroyAllWindows()

```

18.0s

Python

python

```

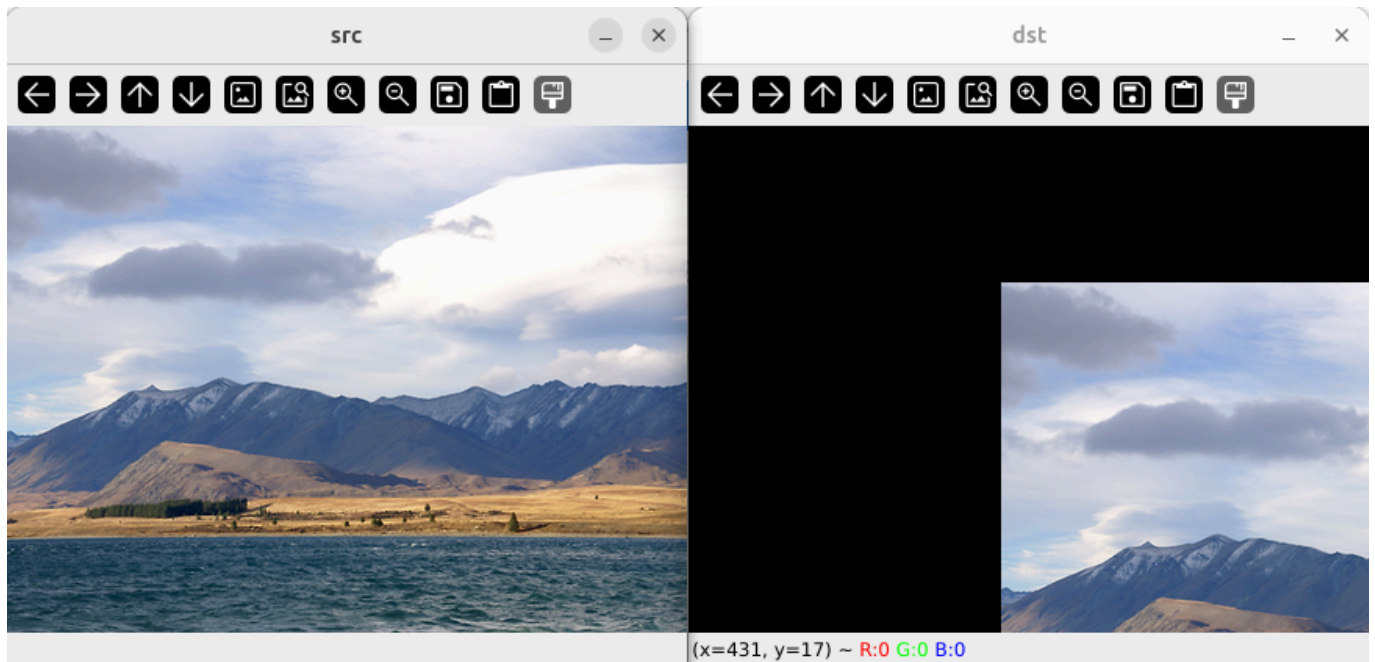
1  src = cv2.imread("./data/tekapo.bmp")
2
3  if src is None:
4      print("Image load failed!")
5      sys.exit()
6
7  aff = np.array([[1, 0, 200],
8                  [0, 1, 100]], dtype=np.float32)
9
10 dst = cv2.warpAffine(src, aff, (0, 0))
11
12 cv2.imshow("src", src)
13 cv2.imshow("dst", dst)
14 cv2.waitKey()
15
16 cv2.destroyAllWindows()

```

노트북 버전과의 차이

- `if src is None:` 로드 실패 검사 추가 → 스크립트 실행 시 필수 습관
- 나머지 로직은 동일 — 행렬 정의 → `warpAffine` → 출력

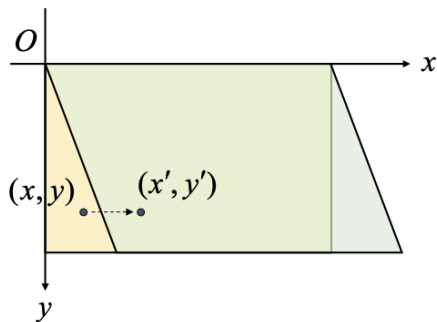
실행 결과 (.py 버전)



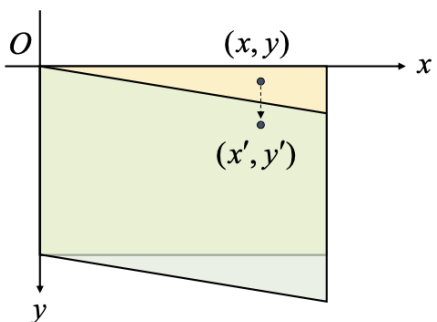
- 노트북 실행 결과와 동일 — 좌·상단 검은 여백, 우·하단 잘림

2. 전단 변환 (Shear)

전단 변환의 수식



$$\begin{cases} x' = x + my \\ y' = y \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & m & 0 \\ 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$



$$\begin{cases} x' = x \\ y' = mx + y \end{cases} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ m & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

핵심 개념

- 전단(shear): 층밀림 — 한 축 방향으로 밀린 정도가 위치에 비례하는 변환
- 가로 방향: $x' = x + m \cdot y$, $y' = y$ → 행렬 $\begin{bmatrix} 1 & m & 0 \\ 0 & 1 & 0 \end{bmatrix}$
- 세로 방향: $x' = x$, $y' = m \cdot x + y$ → 행렬 $\begin{bmatrix} 1 & 0 & 0 \\ m & 1 & 0 \end{bmatrix}$
- 직사각형이 **평행사변형**으로 변형됨(평행성은 유지)

실습 코드 — 전단 변환

```
src = cv2.imread("./data/tekapo.bmp")

aff = np.array([[1, 0.5, 0],
               [0, 1, 0]], dtype=np.float32)

h, w = src.shape[:2]

dst = cv2.warpAffine(src, aff, (w + int(h * 0.5), h))

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

10.3s

Python

python

```
1  src = cv2.imread("./data/tekapo.bmp")
2
3  aff = np.array([[1, 0.5, 0],
4                  [0, 1, 0]], dtype=np.float32)    # 가로 방향 전단, m = 0.5
5
6  h, w = src.shape[:2]
7
8  dst = cv2.warpAffine(src, aff, (w + int(h * 0.5), h))    # 밀린 만큼 폭을 늘림
9
10 cv2.imshow("src", src)
11 cv2.imshow("dst", dst)
12 cv2.waitKey()
13 cv2.destroyAllWindows()
```

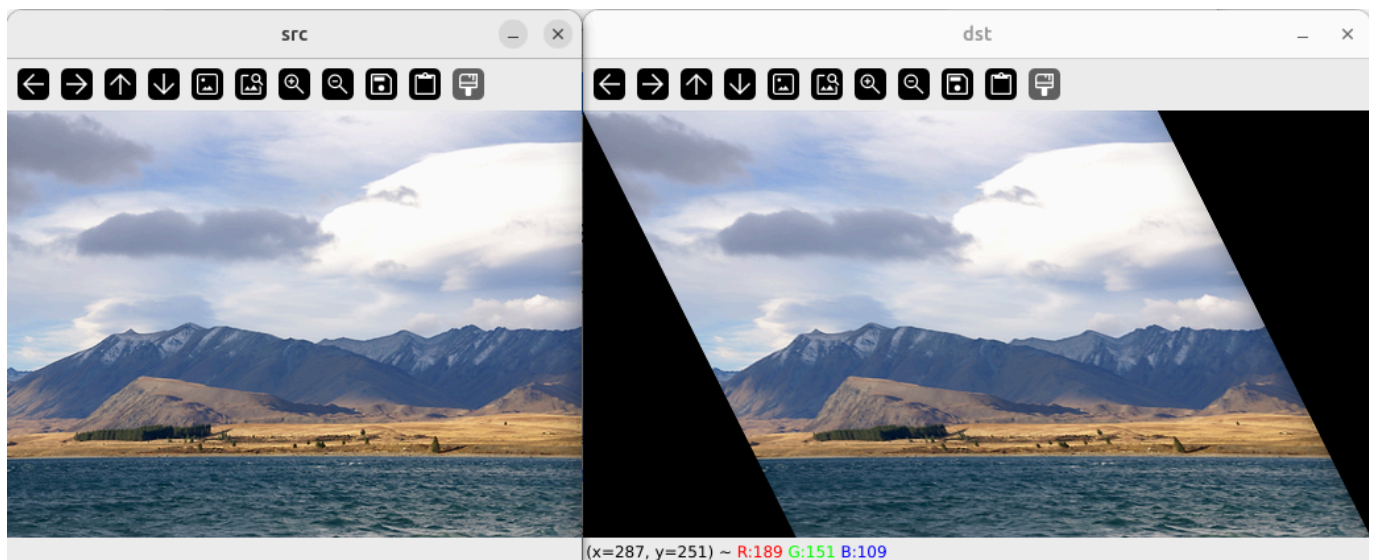
코드 들여다보기 — 사용한 함수·클래스 (2)

이름	종류	설명
----	----	----

<code>np.array([[1, 0.5, 0], [0, 1, 0]])</code>	함수	가로 방향 전단 행렬($m = 0.5$) 생성
<code>src.shape[:2]</code>	속성	(높이 h , 너비 w) 추출 — 출력 크기 계산에 사용
<code>int(h * 0.5)</code>	함수	가장 아래 행이 밀리는 최대 거리 계산 → 정수 변환
<code>cv2.warpAffine(..., (w + int(h*0.5), h))</code>	함수	폭을 늘린 출력 크기 지정 → 밀린 부분까지 담김

`dsize` 를 `(0, 0)` 으로 두면 밀려난 오른쪽이 잘림. 전단·회전처럼 영상 범위가 커지는 변환은 출력 크기를 직접 계산해 주는 것이 요령.

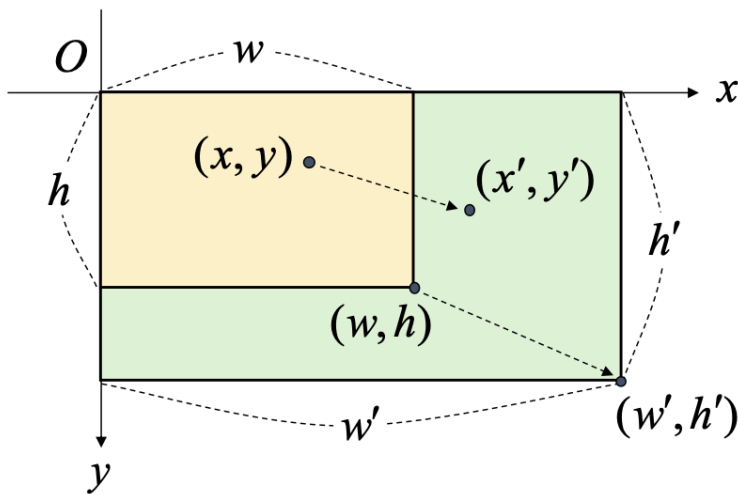
실행 결과 — 전단 변환



- 위쪽은 그대로, 아래로 갈수록 오른쪽으로 밀려 **평행사변형** 형태
- 빈 삼각형 영역은 검은색으로 채워짐

3. 영상의 확대와 축소 (Scaling)

크기 변환의 수식



$$\begin{cases} x' = s_x x \\ y' = s_y y \end{cases} \quad \begin{cases} s_x = w' / w \\ s_y = h' / h \end{cases}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

핵심 개념

- $x' = s_x \cdot x$, $y' = s_y \cdot y$ — 축별 배율을 곱하는 변환
- $s_x = w' / w$, $s_y = h' / h$
- 행렬로는 $[[s_x, 0, 0], [0, s_y, 0]]$ → 역시 어파인 변환의 일종
- $s_x \neq s_y$ 이면 가로세로 비율이 깨짐

cv2.resize() 함수

`cv2.resize(src, dsize, dst=None, fx=None, fy=None, interpolation=None) -> dst`

- src: 입력 영상
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 fx와 fy 값을 이용하여 결정.
- dst: 출력 영상
- fx, fy: x와 y방향 스케일 비율(scale factor). (dsize 값이 0일 때 유효)
- interpolation: 보간법 지정. 기본값은 cv2.INTER_LINEAR

cv2.INTER_NEAREST	최근방 이웃 보간법
cv2.INTER_LINEAR	양선형 보간법 (2x2 이웃 픽셀 참조)
cv2.INTER_CUBIC	3차회선 보간법 (4x4 이웃 픽셀 참조)
cv2.INTER_LANCZOS4	Lanczos 보간법 (8x8 이웃 픽셀 참조)
cv2.INTER_AREA	영상 축소 시 효과적

인자 정리

- `dsize` : 결과 크기 (w, h). `(0, 0)` 이면 `fx · fy` 배율로 결정
- `fx` , `fy` : x·y 방향 스케일 비율 — `dsize`가 0일 때만 유효
- `interpolation` : 보간법. 기본값 `cv2.INTER_LINEAR`
- 확대·축소를 매번 어파인 행렬로 쓰지 않고 **이 함수 하나로** 처리

보간법(interpolation) 비교

플래그	참조 범위	특징	주 용도
<code>INTER_NEAREST</code>	1픽셀	가장 빠름. 계단 현상(에일리어싱) 심함	레이블·마스크 영상
<code>INTER_LINEAR</code>	2×2	속도·품질 균형 — 기본값	일반적인 확대·축소
<code>INTER_CUBIC</code>	4×4	선형보다 부드럽고 선명	품질이 필요한 확대
<code>INTER_LANCZOS4</code>	8×8	가장 좋은 품질, 가장 느림	고품질 확대
<code>INTER_AREA</code>	영역 평균	축소 시 모아레·잡음 억제	영상 축소

실습 코드 — 보간법 4종 비교

```

import sys
import numpy as np
import cv2

src = cv2.imread('./data/rose.bmp') # src.shape=(320, 480)

if src is None:
    print('Image load failed!')
    sys.exit()

dst1 = cv2.resize(src, (0, 0), fx=4, fy=4, interpolation=cv2.INTER_NEAREST)
dst2 = cv2.resize(src, (1920, 1280)) # cv2.INTER_LINEAR
dst3 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_CUBIC)
dst4 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_LANCZOS4)

cv2.imshow('src', src)
cv2.imshow('dst1', dst1[500:900, 400:800])
cv2.imshow('dst2', dst2[500:900, 400:800])
cv2.imshow('dst3', dst3[500:900, 400:800])
cv2.imshow('dst4', dst4[500:900, 400:800])
cv2.waitKey()
cv2.destroyAllWindows()

```

1m 2.3s

Python

python

```

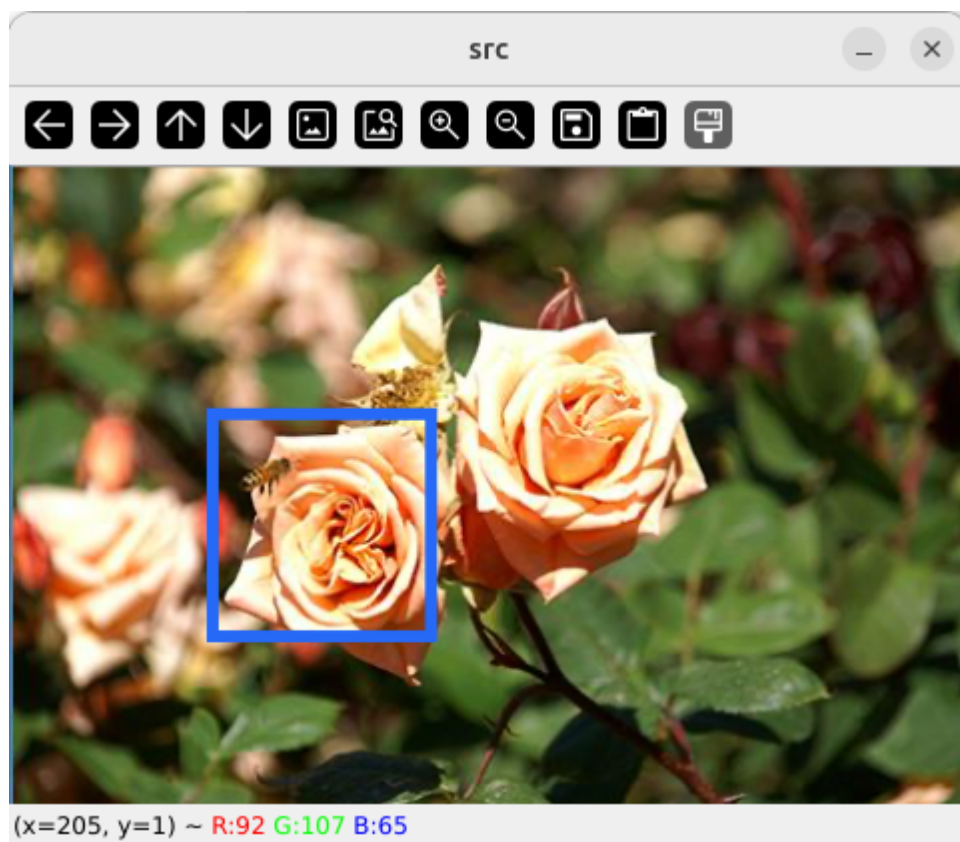
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread('./data/rose.bmp')          # src.shape = (320, 480)
6
7  if src is None:
8      print('Image load failed!')
9      sys.exit()
10
11 dst1 = cv2.resize(src, (0, 0), fx=4, fy=4, interpolation=cv2.INTER_NEAREST)
12 dst2 = cv2.resize(src, (1920, 1280))          # INTER_LINEAR
13 dst3 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_CUBIC)
14 dst4 = cv2.resize(src, (1920, 1280), interpolation=cv2.INTER_LANCZOS4)
15
16 cv2.imshow('src', src)
17 cv2.imshow('dst1', dst1[500:900, 400:800])    # 같은 영역만 잘라 비교
18 cv2.imshow('dst2', dst2[500:900, 400:800])
19 cv2.imshow('dst3', dst3[500:900, 400:800])
20 cv2.imshow('dst4', dst4[500:900, 400:800])
21 cv2.waitKey()
22 cv2.destroyAllWindows()

```

코드 들여다보기 — 사용한 함수·클래스 (3)

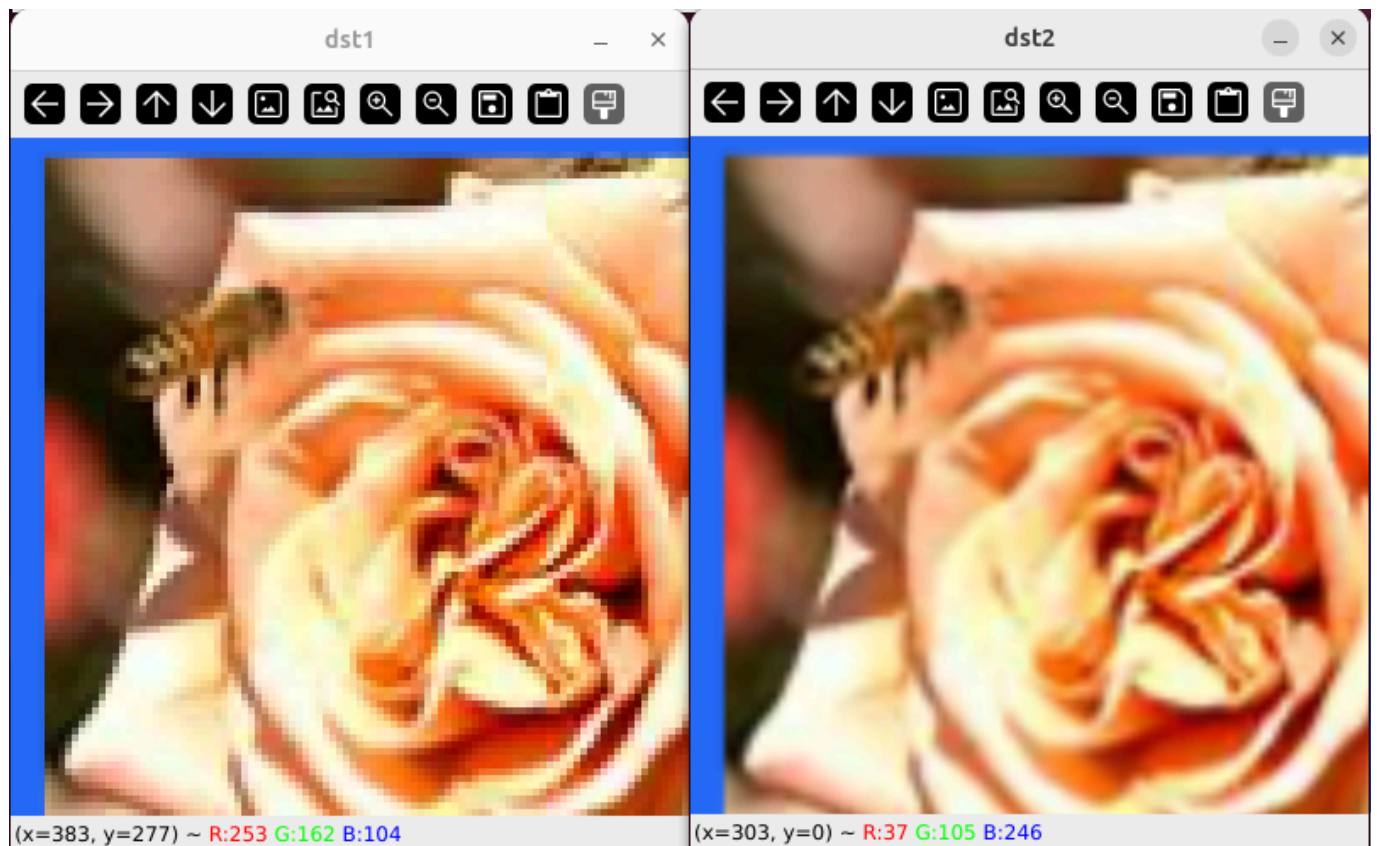
이름	종류	설명
<code>cv2.resize(src, (0,0), fx=4, fy=4)</code>	함수	dsize를 0으로 두고 배율로 4배 확대
<code>cv2.resize(src, (1920, 1280))</code>	함수	결과 크기를 직접 지정(보간법 생략 → <code>INTER_LINEAR</code>)
<code>cv2.INTER_NEAREST</code>	상수	최근방 이웃 보간 — 계단 현상 확인용
<code>cv2.INTER_CUBIC</code>	상수	3차 회선 보간(4×4 참조)
<code>cv2.INTER_LANCZOS4</code>	상수	Lanczos 보간(8×8 참조) — 최고 품질
<code>dst1[500:900, 400:800]</code>	슬라이싱	확대 영상의 같은 위치를 잘라 보간 품질만 비교

원본 영상과 비교 구간



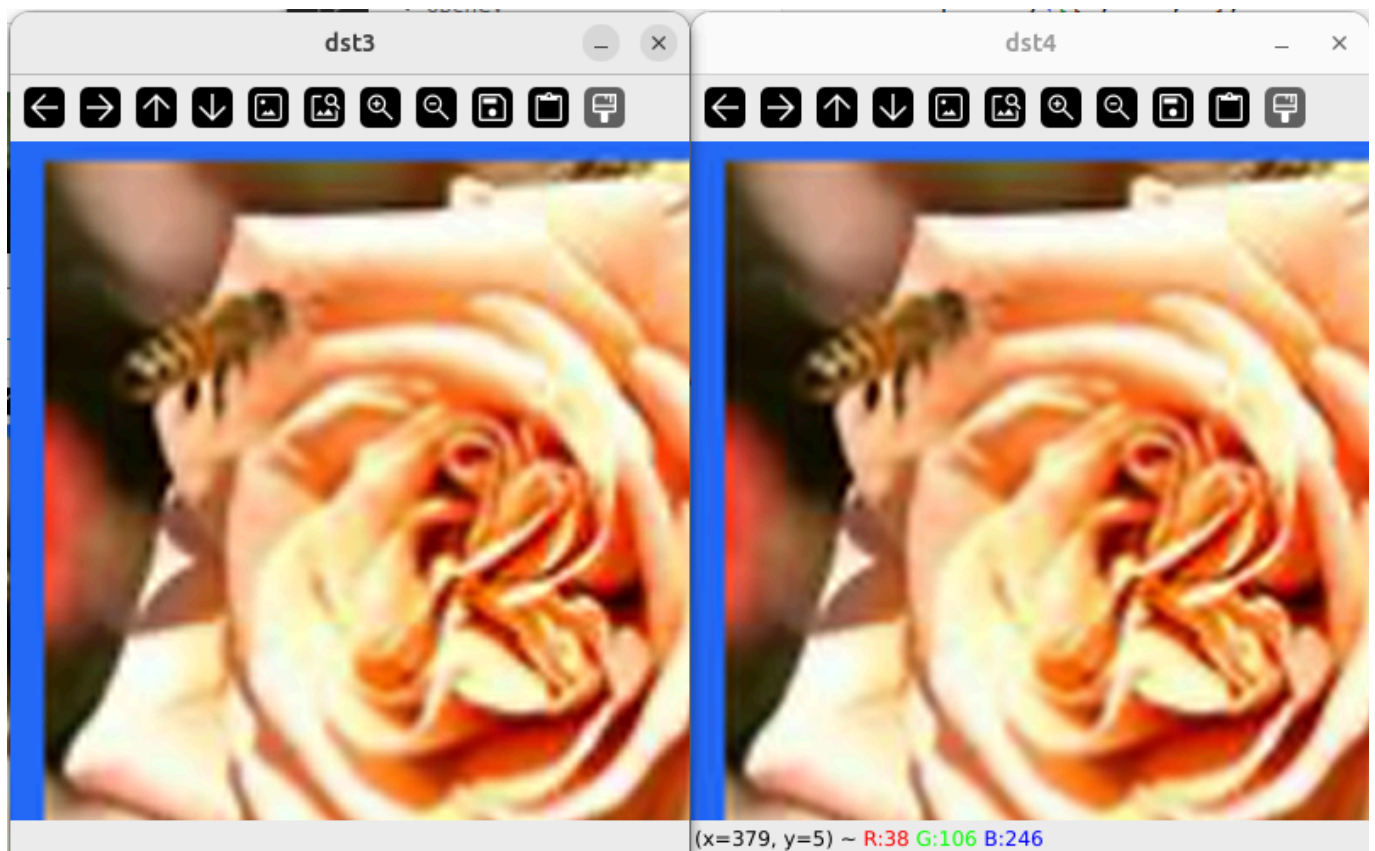
- 원본 `rose.bmp(480×320)`에서 파란 사각형 영역이 4배 확대 후 비교할 구간

실행 결과 — NEAREST vs LINEAR



- **dst1** (NEAREST): 화소가 사각형 블록으로 뭉쳐 계단 현상이 뚜렷
- **dst2** (LINEAR): 부드럽지만 다소 흐릿함

실행 결과 — CUBIC vs LANCZOS4



- `dst3` (CUBIC) · `dst4` (LANCZOS4): 선형보다 윤곽이 또렷
- 차이는 크지 않지만 연산 시간은 뚜렷하게 증가 → 품질과 속도의 트레이드오프

대칭 변환 — `cv2.flip()`

```
cv2.flip(src, flipCode, dst=None) -> dst
```

- `src`: 입력 영상
- `flipCode`: 대칭 방향 지정

양수 (+1)	좌우 대칭
0	상하 대칭
음수 (-1)	좌우 & 상하 대칭

- `dst`: 출력 영상

사용 포인트

- `flipCode` 양수(+1): 좌우 대칭 / 0: 상하 대칭 / 음수(-1): 좌우 + 상하
- 행렬 연산 없이 인덱스만 뒤집는 방식이라 매우 빠름
- 딥러닝 데이터 증강에서 가장 자주 쓰이는 변환 중 하나

영상 피라미드 — `cv2.pyrDown()`

```
cv2.pyrDown(src, dst=None, dstsize=None, borderType=None) -> dst
```

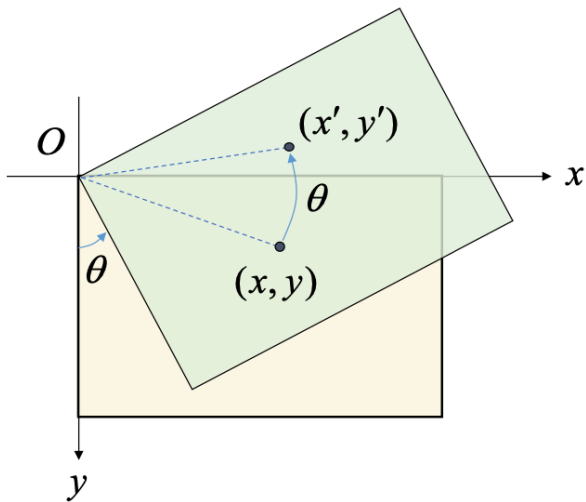
- `src`: 입력 영상
- `dst`: 출력 영상
- `dstsize`: 출력 영상 크기.
따로 지정하지 않으면 입력 영상의 가로, 세로 크기의 1/2로 설정.
- `borderType`: 가장자리 픽셀 확장 방식
- 참고 사항
 - 먼저 5x5 크기의 가우시안 필터를 적용
 - 이후 짝수 행과 열을 제거하여 작은 크기의 영상을 생성

사용 포인트

- `dstsize` 를 생략하면 가로·세로가 각각 1/2(면적 1/4)로 축소
- 동작 순서: 5x5 가우시안 필터 적용 → 짝수 행·열 제거
- 미리 흐리게 만든 뒤 속아 내므로 `resize` 만 쓸 때보다 계단 현상·모아레가 적음
- 반대 방향 확대는 `cv2.pyrUp()` — 다중 스케일 검출의 기본 도구

4. 회전 (Rotation)

회전 변환의 수식



$$\begin{cases} x' = \cos \theta \cdot x + \sin \theta \cdot y \\ y' = -\sin \theta \cdot x + \cos \theta \cdot y \end{cases}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos \theta & \sin \theta & 0 \\ -\sin \theta & \cos \theta & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

핵심 개념

- 원점 기준 회전: $x' = \cos\theta \cdot x + \sin\theta \cdot y$, $y' = -\sin\theta \cdot x + \cos\theta \cdot y$
- 행렬은 $\begin{bmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \end{bmatrix}$
- 영상 좌표계는 y축이 아래로 향하므로 수학 좌표계와 부호 배치가 다름
- 기준점이 원점(좌측 상단) → 그대로 쓰면 영상이 화면 밖으로 크게 벗어남

실습 코드 — 원점 기준 회전(20°)

```
import math

src = cv2.imread("./data/tekapo.bmp")

rad = 20 * math.pi / 180
aff = np.array([[math.cos(rad), math.sin(rad), 0],
                [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)

dst = cv2.warpAffine(src, aff, (0, 0))

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()
cv2.destroyAllWindows()
```

python

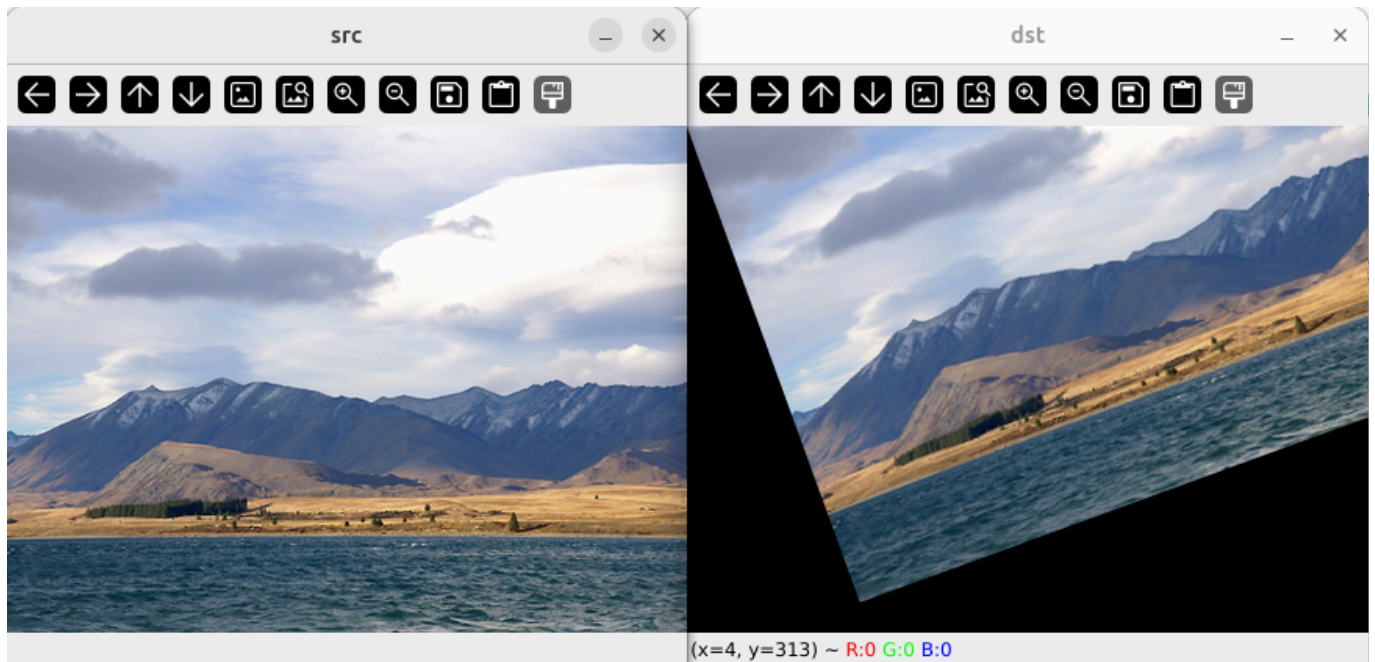
```
1 import math
2
3 src = cv2.imread("./data/tekapo.bmp")
4
5 rad = 20 * math.pi / 180 # degree → radian
6 aff = np.array([[ math.cos(rad), math.sin(rad), 0],
7                 [-math.sin(rad), math.cos(rad), 0]], dtype=np.float32)
8
9 dst = cv2.warpAffine(src, aff, (0, 0))
10
11 cv2.imshow("src", src)
12 cv2.imshow("dst", dst)
13 cv2.waitKey()
14 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (4)

이름	종류	설명
math	모듈	삼각함수·원주율 제공 표준 모듈
math.pi	상수	원주율 — 각도를 라디안으로 바꿀 때 사용
math.cos() / math.sin()	함수	회전 행렬의 코사인·사인 항 계산(인자는 라디안)
np.array(..., np.float32)	함수	2×3 회전 행렬 생성
cv2.warpAffine()	함수	회전 행렬 적용

파이썬 삼각함수는 라디안을 받음 → $\text{도} \times \pi / 180$ 변환이 필요. `math.radians(20)` 으로도 동일하게 계산 가능.

실행 결과 — 원점 기준 회전의 한계



- 좌측 상단(0, 0)을 축으로 돌아가 영상 대부분이 화면 밖으로 밀려남
- 실무에서 원하는 것은 대개 영상 중앙 기준 회전 → 다음 함수가 필요한 이유

cv2.getRotationMatrix2D() 함수

```
cv2.getRotationMatrix2D(center, angle, scale) -> retval
```

- center: 회전 중심 좌표. (x, y) 튜플.
- angle: (반시계 방향) 회전 각도(degree). 음수는 시계 방향.
- scale: 추가적인 확대 비율
- retval: 2x3 어파인 변환 행렬. 실수형.

$$\begin{bmatrix} \alpha & \beta & (1-\alpha) \cdot \text{center.x} - \beta \cdot \text{center.y} \\ -\beta & \alpha & \beta \cdot \text{center.x} + (1-\alpha) \cdot \text{center.y} \end{bmatrix} \text{ where } \begin{cases} \alpha = \text{scale} \cdot \cos(\text{angle}) \\ \beta = \text{scale} \cdot \sin(\text{angle}) \end{cases}$$

인자 정리

- `center` : 회전 중심 (x, y) 튜플 — 보통 영상 중앙
- `angle` : 반시계 방향 회전 각도(도 단위). 음수면 시계 방향
- `scale` : 회전과 동시에 적용할 확대 비율
- 반환값: 2×3 어파인 변환 행렬(실수형) → 그대로 `warpAffine` 에 전달
- 행렬 내부에 이동 항이 포함되어 중심 이동까지 한 번에 처리됨

실습 코드 — 중앙 기준 회전 + 축소

```
import sys
import numpy as np
import cv2

src = cv2.imread("./data/tekapo.bmp")

if src is None:
    print("Image load failed!")
    sys.exit()

cp = (src.shape[1] / 2, src.shape[0] / 2)
rot = cv2.getRotationMatrix2D(cp, 20, 0.7)

dst = cv2.warpAffine(src, rot, (0, 0))

cv2.imshow("src", src)
cv2.imshow("dst", dst)
cv2.waitKey()

cv2.destroyAllWindows()
```

9.7s

Python

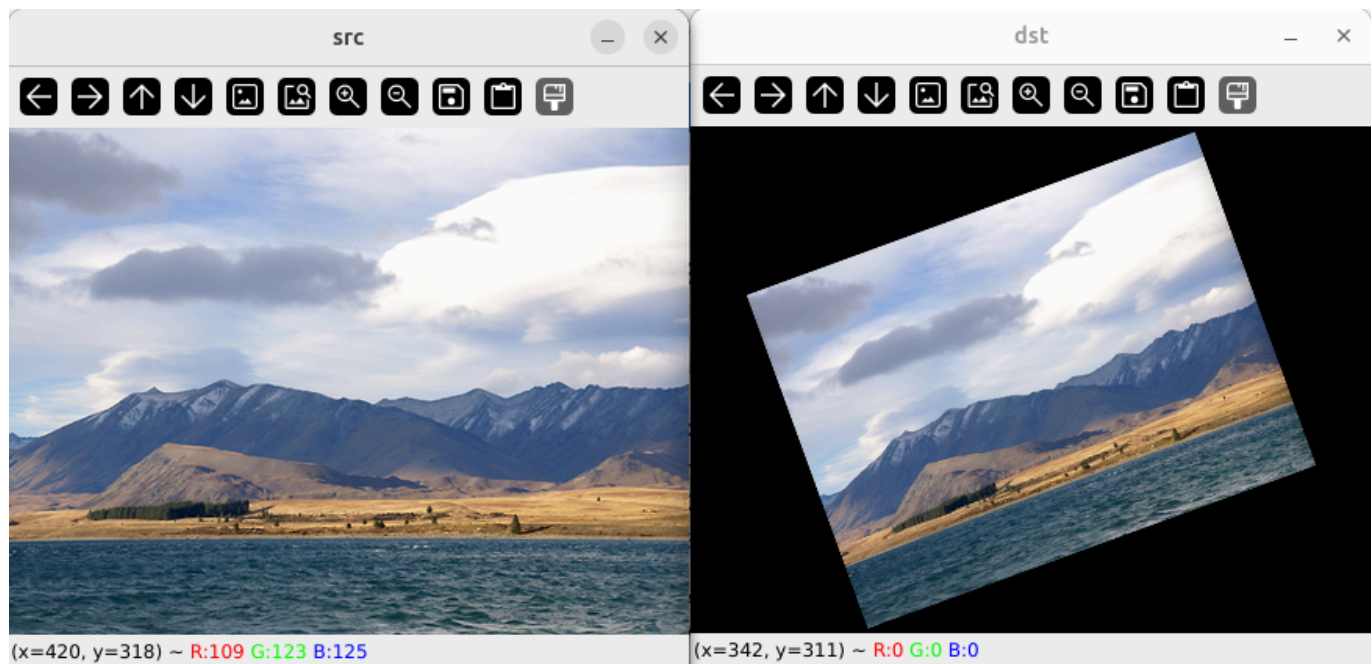
python

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread("./data/tekapo.bmp")
6
7  if src is None:
8      print("Image load failed!")
9      sys.exit()
10
11  cp = (src.shape[1] / 2, src.shape[0] / 2)      # 영상 중앙 (x, y)
12  rot = cv2.getRotationMatrix2D(cp, 20, 0.7)    # 20도 회전 + 0.7배 축소
13
14  dst = cv2.warpAffine(src, rot, (0, 0))
15
16  cv2.imshow("src", src)
17  cv2.imshow("dst", dst)
18  cv2.waitKey()
19
20  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (5)

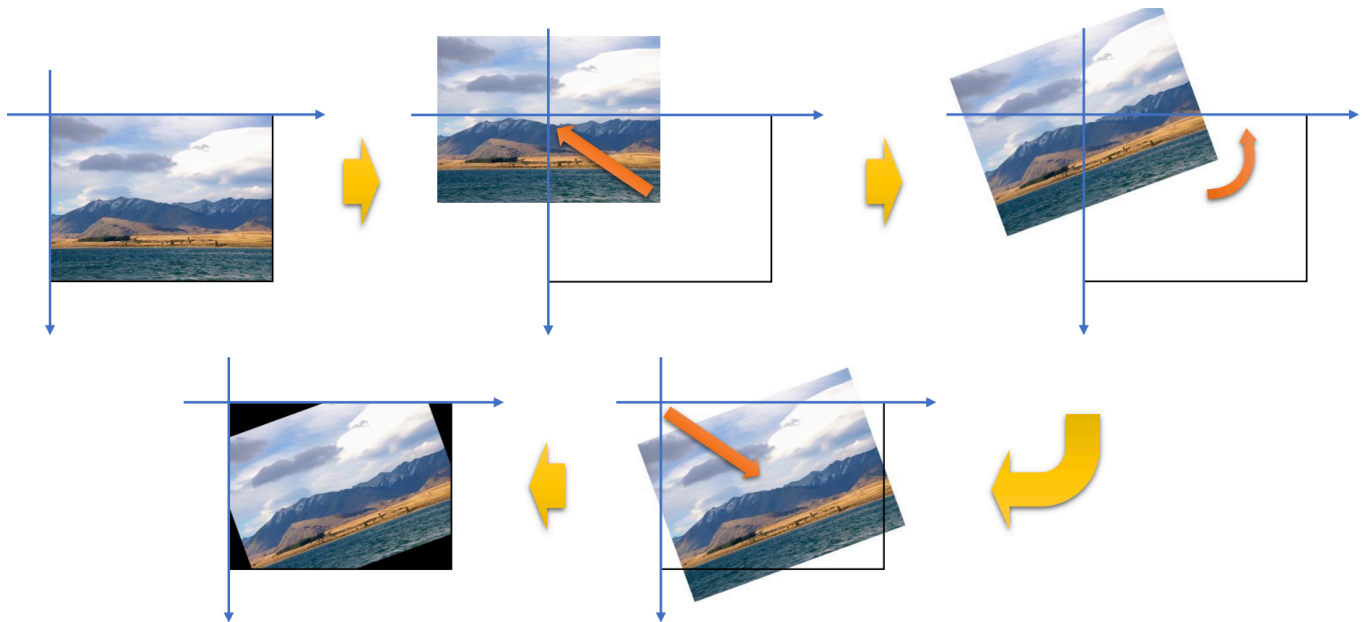
이름	종류	설명
src.shape[1] / 2, src.shape[0] / 2	연산	너비/2, 높이/2 → 영상 중앙 좌표. 순서는 (x, y)
cv2.getRotationMatrix2D()	함수	중심·각도·배율로부터 2×3 회전 행렬 생성
20	인자	반시계 방향 20도
0.7	인자	0.7배 축소 → 회전 후 잘림을 줄이는 효과
cv2.warpAffine()	함수	생성된 회전 행렬 적용

실행 결과 — 중앙 기준 회전



- 영상 중앙을 축으로 반시계 20도 회전 + 0.7배 축소
- 축소 덕분에 네 모서리가 화면 안에 들어옴 — 잘림 없이 전체가 보임

직접 구현한다면 — 이동·회전·역이동

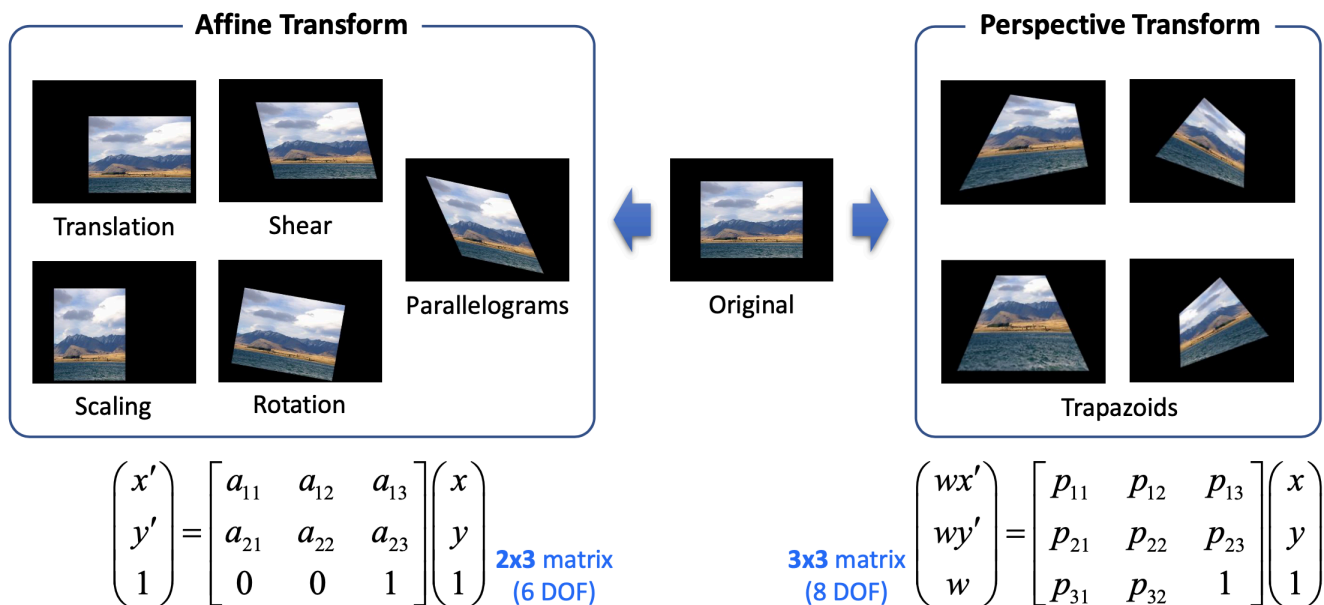


행렬 3개의 합성

- ① 회전 중심을 원점으로 이동(-cx, -cy)
- ② 원점 기준으로 회전
- ③ 다시 원래 위치로 이동(+cx, +cy)
- 세 행렬을 곱한 결과가 곧 `getRotationMatrix2D` 가 만들어 주는 행렬
- 임의의 축을 기준으로 하는 변환은 모두 이 이동 → 변환 → 역이동 패턴

5. 어파인 변환과 투시 변환

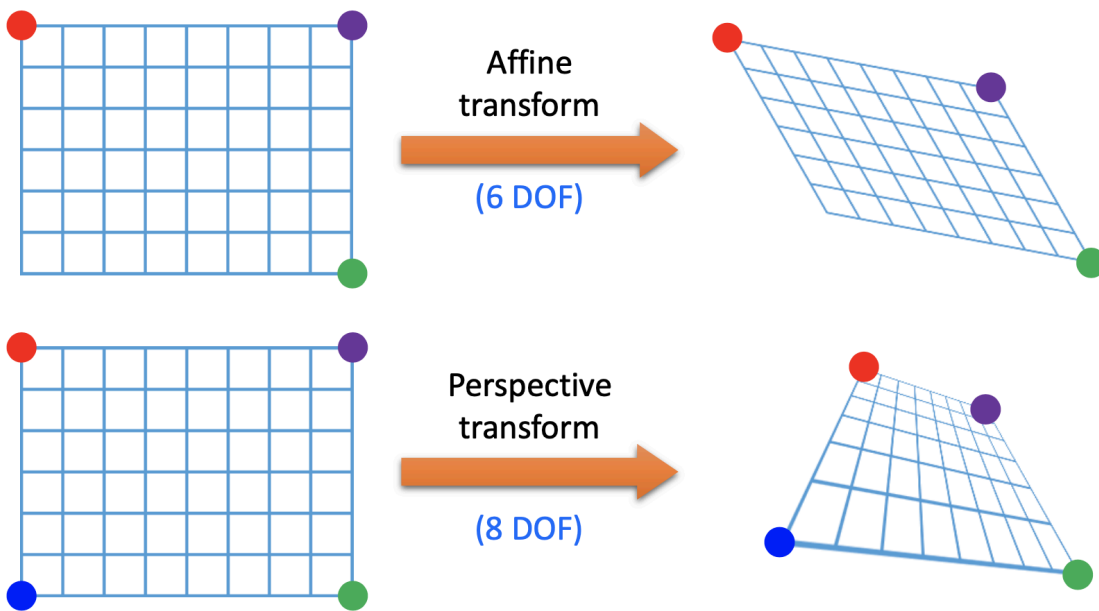
두 변환의 비교



핵심 개념

- **어파인 변환**: 이동·전단·크기·회전을 모두 포함. **2x3 행렬, 6 자유도**
- 결과는 항상 **평행사변형** — 원래 평행하던 변은 변환 후에도 평행
- **투시 변환**: 원근(perspective)까지 표현. **3x3 행렬, 8 자유도**
- 결과는 **사다리꼴** — 평행성이 깨지고 멀리 있는 쪽이 좁아짐
- 투시 변환은 마지막에 **w** 로 나누는 과정이 있어 **비선형**

기하학적 차이



- 어파인(6 DOF): 대응점 **3쌍**이면 행렬이 결정됨(격자가 평행사변형 유지)
- 투시(8 DOF): 대응점 **4쌍**이 필요(격자가 원근에 따라 좁아짐)
- 자유도 = 필요한 대응점 개수 $\times 2$ (한 점이 x·y 두 방정식을 제공)

cv2.getAffineTransform() 함수

```
cv2.getAffineTransform(src, dst) -> retval
```

- src: 3개의 원본 좌표점. numpy.ndarray. shape=(3, 2)
e.g) np.array([[x₁, y₁], [x₂, y₂], [x₃, y₃]], np.float32)
- dst: 3개의 결과 좌표점. numpy.ndarray. shape=(3, 2)
- retval: 2x3 투시 변환 행렬

$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \text{map_matrix} \cdot \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where $dst(i) = (x'_i, y'_i), src(i) = (x_i, y_i), i = 0, 1, 2$

사용 포인트

- `src` · `dst` : 3개의 좌표점. shape=(3, 2), `np.float32`
- 반환값: 2×3 어파인 변환 행렬 → `warpAffine` 에 전달
- 3쌍의 대응점만 알면 이동·회전·크기·전단이 섞인 변환을 한 번에 구할 수 있음

원본 슬라이드의 `retval` 설명에 '2x3 투시 변환 행렬'로 적혀 있으나, 정확히는 2×3 어파인 변환 행렬임(투시는 3×3).

cv2.getPerspectiveTransform() 함수

```
cv2.getPerspectiveTransform(src, dst, solveMethod=None) -> retval
```

- `src`: 4개의 원본 좌표점. numpy.ndarray. shape=(4, 2)
e.g) `np.array([[x1, y1], [x2, y2], [x3, y3], [x4, y4]], np.float32)`
- `dst`: 4개의 결과 좌표점. numpy.ndarray. shape=(4, 2)
- `retval`: 3x3 투시 변환 행렬

$$\begin{bmatrix} t_i x'_i \\ t_i y'_i \\ t_i \end{bmatrix} = \text{map_matrix} \cdot \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

where $dst(i) = (x'_i, y'_i), src(i) = (x_i, y_i), i = 0, 1, 2, 3$

사용 포인트

- `src` · `dst` : 4개의 좌표점. shape=(4, 2), `np.float32`
- 반환값: 3×3 투시 변환 행렬 → `warpPerspective` 에 전달
- 네 점의 대응 순서가 반드시 일치해야 함(좌상 → 좌하 → 우하 → 우상 등 일관된 순서)

cv2.warpAffine() 다시 보기

```
cv2.warpAffine(src, M, dsize, dst=None, flags=None,
               borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 2x3 어파인 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR.
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

- 2x3 행렬 **M** 과 출력 크기 **dsize** 가 핵심 인자
- 어파인 계열(이동·전단·크기·회전)은 모두 이 함수로 적용

cv2.warpPerspective() 함수

```
cv2.warpPerspective(src, M, dsize, dst=None, flags=None,
                    borderMode=None, borderValue=None) -> dst
```

- src: 입력 영상
- M: 3x3 투시 변환 행렬. 실수형.
- dsize: 결과 영상 크기. (w, h) 튜플. (0, 0)이면 src와 같은 크기로 설정.
- dst: 출력 영상
- flags: 보간법. 기본값은 cv2.INTER_LINEAR
- borderMode: 가장자리 픽셀 확장 방식. 기본값은 cv2.BORDER_CONSTANT.
- borderValue: cv2.BORDER_CONSTANT일 때 사용할 상수 값. 기본값은 0.

사용 포인트

- **M** 은 3x3 투시 변환 행렬 — warpAffine과의 유일한 형태 차이
- 나머지 인자(**dsize** · **flags** · **borderMode**)는 warpAffine과 동일

어파인 vs 투시 정리

항목	어파인 변환	투시 변환
행렬 크기	2×3	3×3
자유도	6	8
필요한 대응점	3쌍	4쌍
행렬 생성	<code>cv2.getAffineTransform()</code>	<code>cv2.getPerspectiveTransform()</code>
적용 함수	<code>cv2.warpAffine()</code>	<code>cv2.warpPerspective()</code>
결과 형태	평행사변형(평행성 유지)	사다리꼴(평행성 깨짐)
대표 용도	회전·확대·기울이기, 데이터 증강	문서 스캔, 노면 조감도(bird eye view)

6. 응용 — 명함 펴기와 문서 스캐너

실습 코드 — 명함 펴기

```
import sys
import numpy as np
import cv2

src = cv2.imread('./data/pinkwink_namecard.png')

if src is None:
    print('Image load failed!')
    sys.exit()

w, h = 720, 400
srcQuad = np.array([[360, 345], [879, 404], [895, 664], [254, 573]], np.float32)
dstQuad = np.array([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]], np.float32)

pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)
dst = cv2.warpPerspective(src, pers, (w, h))

cv2.imshow('src', src)
cv2.imshow('dst', dst)
while True:
    if cv2.waitKey() == 27:
        break

cv2.destroyAllWindows()
```

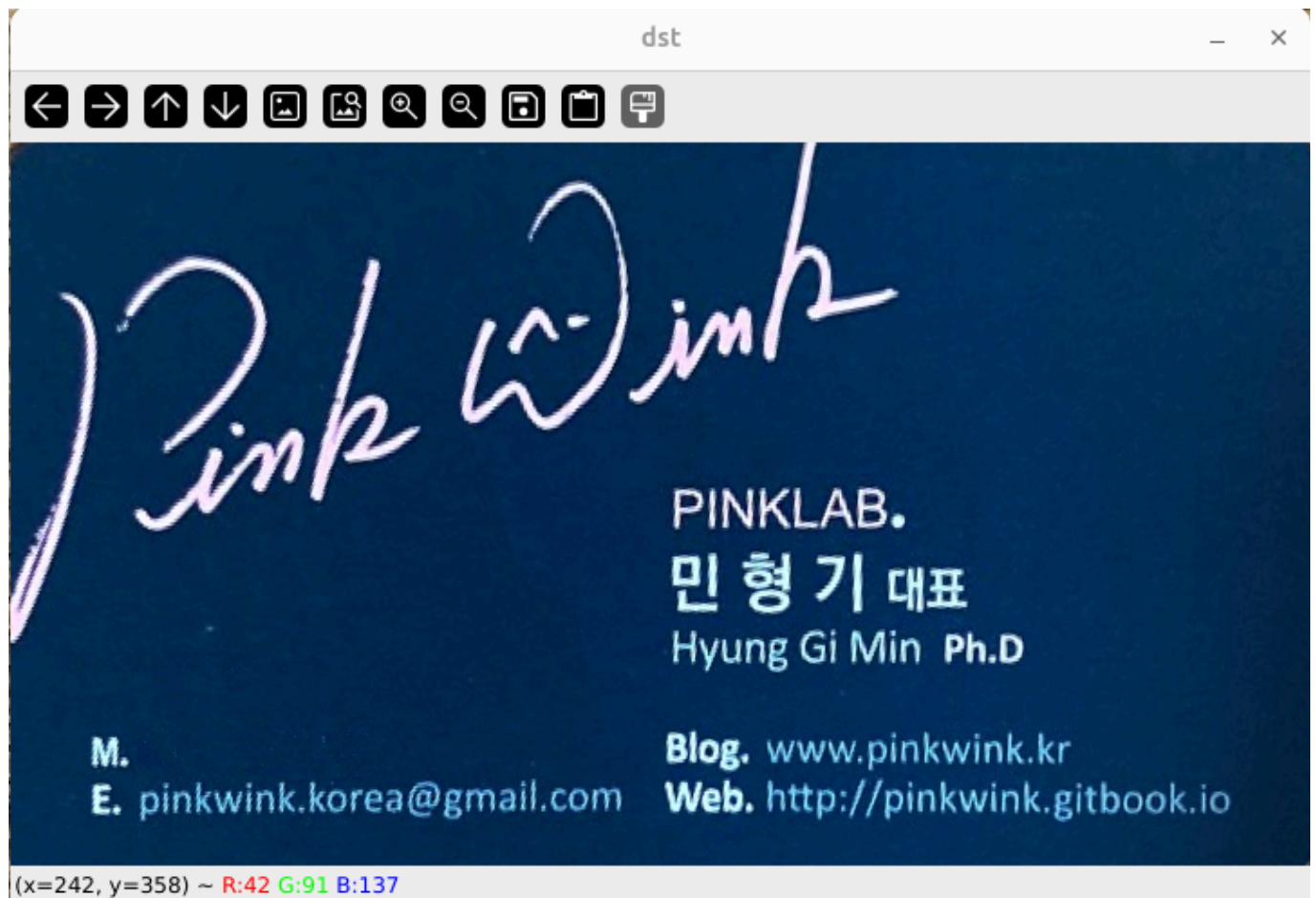
python

```
1  import sys
2  import numpy as np
3  import cv2
4
5  src = cv2.imread('./data/pinkwink_namecard.png')
6
7  if src is None:
8      print('Image load failed!')
9      sys.exit()
10
11  w, h = 720, 400                                # 펴서 만들 결과 영상 크기
12
13  srcQuad = np.array([[360, 345], [879, 404], [895, 664], [254, 573]], np.float32)
14  dstQuad = np.array([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]], np.float32)
15
16  pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)
17  dst = cv2.warpPerspective(src, pers, (w, h))
18
19  cv2.imshow('src', src)
20  cv2.imshow('dst', dst)
21  while True:
22      if cv2.waitKey() == 27:                    # ESC로 종료
23          break
24
25  cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (6)

이름	종류	설명
srcQuad	변수	원본에서 명함 네 모서리의 좌표(좌상 → 우상 → 우하 → 좌하 순)
dstQuad	변수	펴진 결과에서의 네 모서리 — 폭 w, 높이 h인 직사각형
np.array(..., np.float32)	함수	좌표 배열 생성. shape=(4, 2), 실수형 필수
cv2.getPerspectiveTransform()	함수	네 쌍의 대응점으로 3×3 투시 변환 행렬 계산
cv2.warpPerspective()	함수	구한 행렬로 영상을 펴서 (w, h) 크기로 출력
w-1, h-1	연산	좌표는 0부터 시작하므로 마지막 픽셀 인덱스는 크기-1
cv2.waitKey() == 27	조건	ESC 키(아스키 27) 입력 시 루프 종료

실행 결과 — 명함 펴기

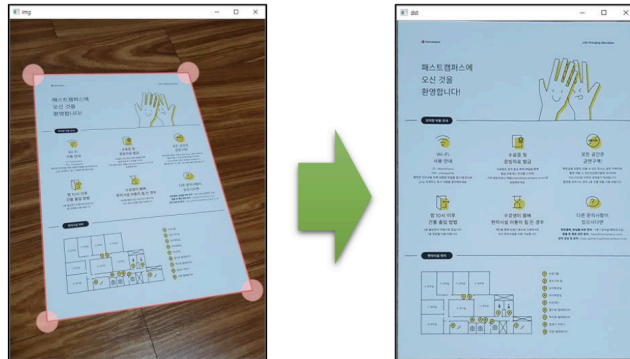


- 비스듬히 찍힌 명함이 정면에서 본 직사각형으로 퍼짐
- 네 모서리 좌표만 알면 원근 왜곡을 제거할 수 있음 → OCR 전처리의 표준 절차
- 좌표를 손으로 입력했다는 점이 한계 → 다음 예제에서 마우스로 지정

문서 스캐너 — 아이디어

■ 왜곡된 문서 영상을 직사각형 형태로 똑바로 펴기 (투시 변환)

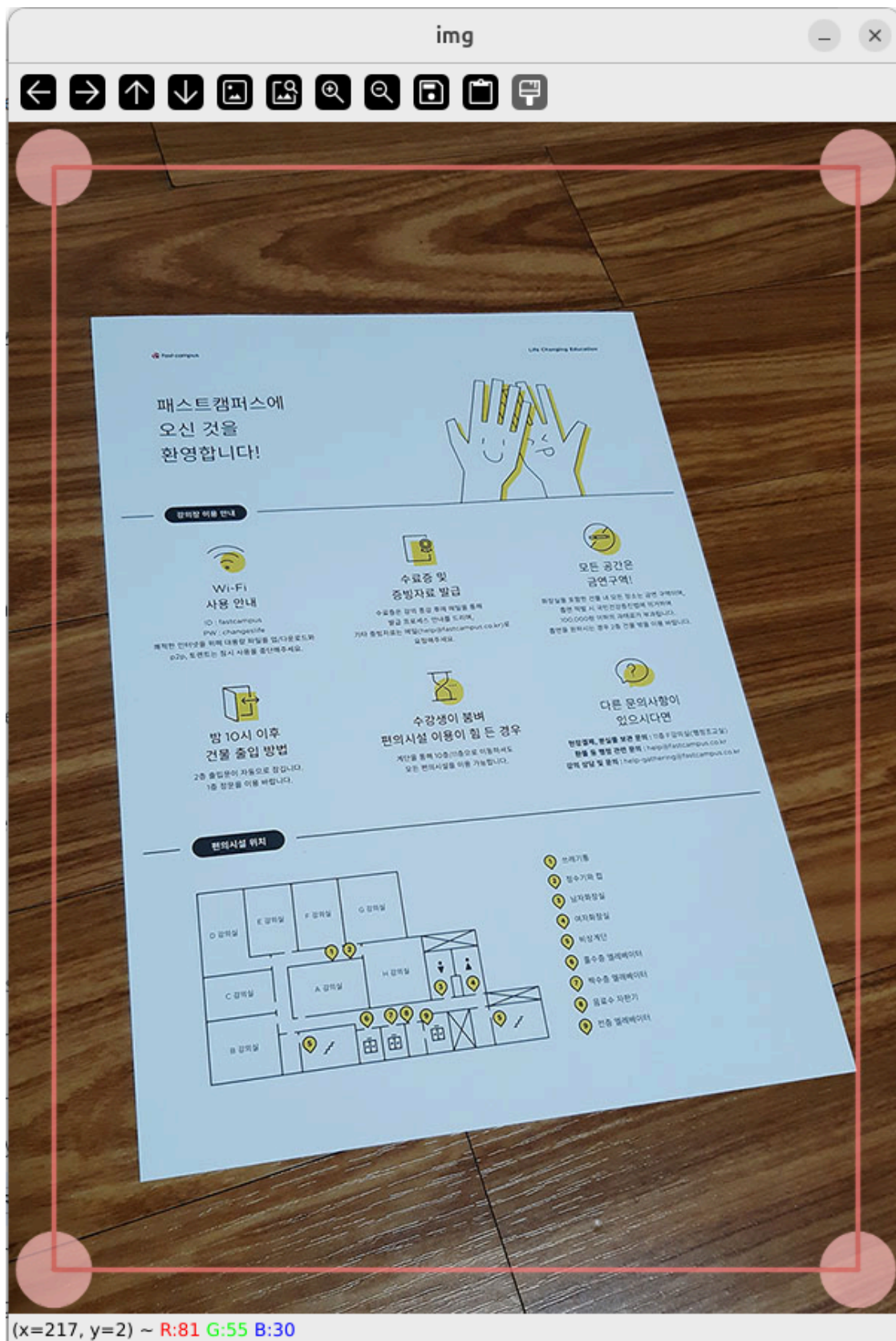
- 네 개의 모서리 좌표를 순서대로 srcQuad 배열에 추가
- dstQuad 배열에는 미리 정의한 A4 용지 크기의 네 모서리 좌표를 저장 (A4 용지 크기: 210x297cm)
- srcQuad 점들로부터 dstQuad 점들로 이동하는 투시 변환 계산
- 계산된 투시 변환 행렬을 이용하여 영상을 투시 변환하여 화면 출력



처리 흐름

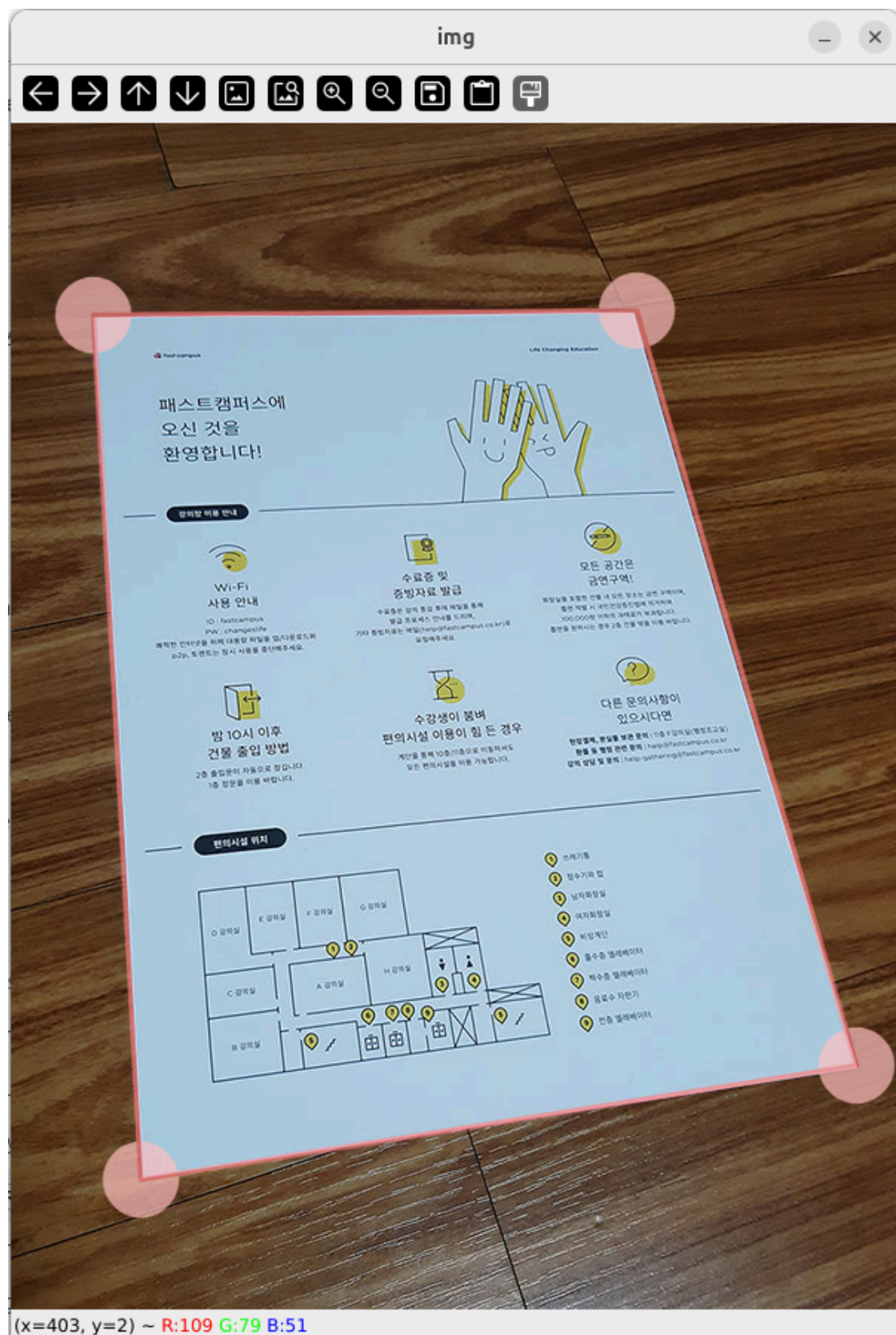
- ① 네 모서리 좌표를 순서대로 **srcQuad** 에 저장
- ② A4 비율(210×297)로 계산한 결과 크기의 네 모서리를 **dstQuad** 에 저장
- ③ 두 좌표 집합으로 투시 변환 행렬 계산
- ④ 변환을 적용해 똑바로 펴진 문서 출력

실행 화면 (1) — 초기 사각형



- 영상 가장자리에서 30픽셀 안쪽에 초기 사각형과 네 모서리 원이 표시됨
- 아직 문서 모서리와 맞지 않은 상태

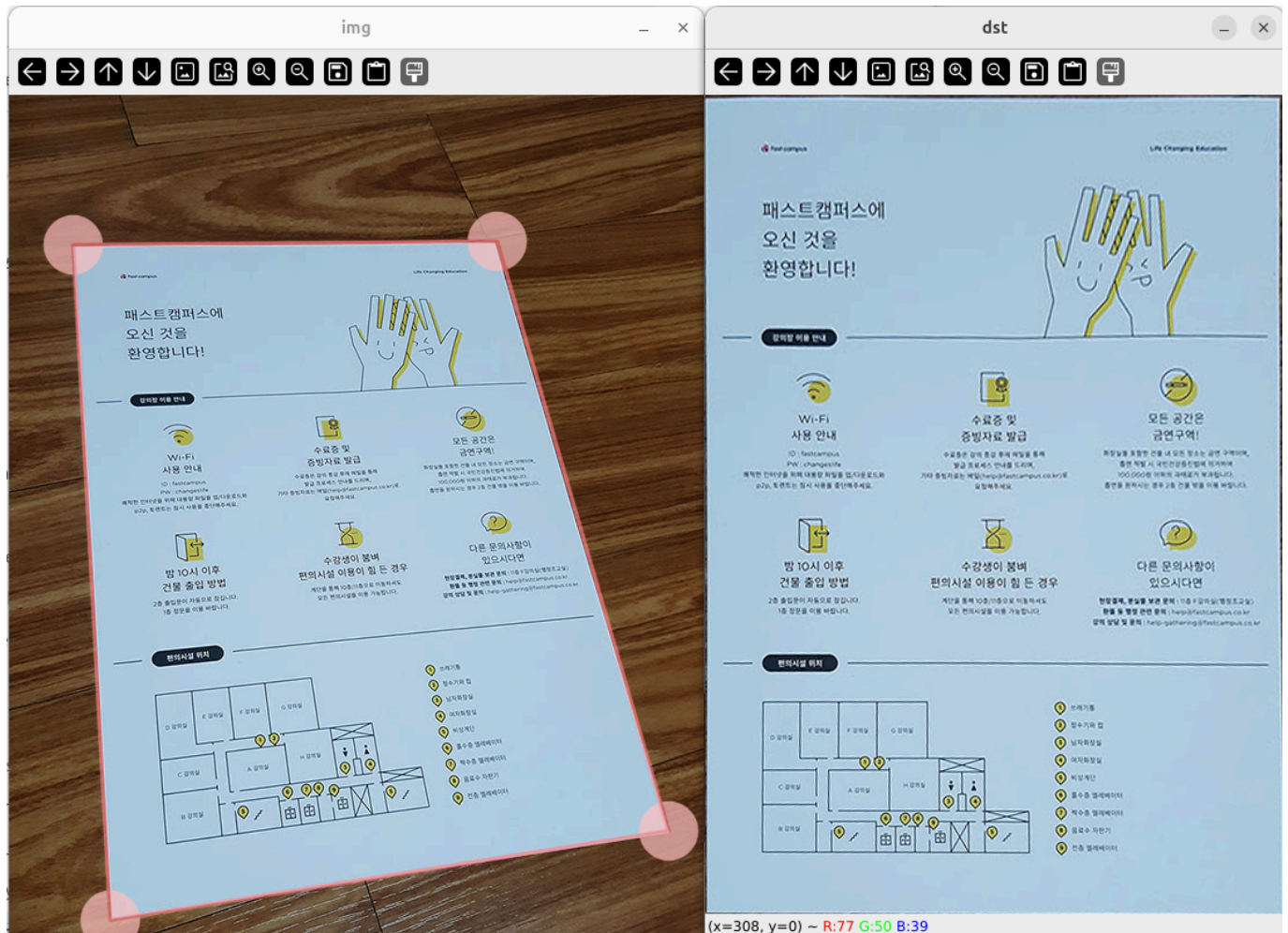
실행 화면 (2) — 마우스로 모서리 맞추기



(x=403, y=2) ~ R:109 G:79 B:51

- 각 모서리 원을 드래그해 문서 네 귀퉁이에 맞춤
- 드래그 중에도 사각형이 실시간으로 다시 그려짐
- 맞춘 뒤 **ENTER**를 누르면 투시 변환 수행

실행 화면 (3) — 변환 결과



- 기울어져 찍힌 안내문이 스캔한 것처럼 정면으로 펴짐
- 결과 크기는 A4 비율(가로 500 → 세로 약 707)로 지정

실습 코드 (1) — ROI 그리기 함수

openCV > 13_geometrical_transfer_01.py > onMouse

```
1  import sys
2  import numpy as np
3  import cv2
4
5  def drawROI(img, corners):
6      cpy = img.copy()
7
8      c1 = (192, 192, 255)
9      c2 = (128, 128, 255)
10
11     for pt in corners:
12         cv2.circle(cpy, tuple(pt.astype(int)), 25, c1, -1, cv2.LINE_AA)
13
14     cv2.line(
15         cpy,
16         tuple(corners[0].astype(int)),
17         tuple(corners[1].astype(int)),
18         c2,
19         2,
20         cv2.LINE_AA,
21     )
22     cv2.line(
23         cpy,
24         tuple(corners[1].astype(int)),
25         tuple(corners[2].astype(int)),
26         c2,
27         2,
28         cv2.LINE_AA,
29     )
30     cv2.line(
31         cpy,
32         tuple(corners[2].astype(int)),
33         tuple(corners[3].astype(int)),
34         c2,
35         2,
36         cv2.LINE_AA,
37     )
38     cv2.line(
39         cpy,
40         tuple(corners[3].astype(int)),
41         tuple(corners[0].astype(int)),
42         c2,
43         2,
44         cv2.LINE_AA,
45     )
46
47     disp = cv2.addWeighted(img, 0.3, cpy, 0.7, 0)
48
49     return disp
```

python

```
1 import sys
2 import numpy as np
3 import cv2
4
5
6 def drawROI(img, corners):
7     cpy = img.copy()
8
9     c1 = (192, 192, 255)
10    c2 = (128, 128, 255)
11
12    for pt in corners:
13        cv2.circle(cpy, tuple(pt.astype(int)), 25, c1, -1, cv2.LINE_AA)
14
15        cv2.line(cpy, tuple(corners[0].astype(int)), tuple(corners[1].astype(int)), c2,
16                2, cv2.LINE_AA)
17        cv2.line(cpy, tuple(corners[1].astype(int)), tuple(corners[2].astype(int)), c2,
18                2, cv2.LINE_AA)
19        cv2.line(cpy, tuple(corners[2].astype(int)), tuple(corners[3].astype(int)), c2,
20                2, cv2.LINE_AA)
21        cv2.line(cpy, tuple(corners[3].astype(int)), tuple(corners[0].astype(int)), c2,
22                2, cv2.LINE_AA)
23
24    disp = cv2.addWeighted(img, 0.3, cpy, 0.7, 0)
25
26    return disp
```

코드 들여다보기 — 사용한 함수·클래스 (7)

이름	종류	설명
drawROI()	함수	네 모서리 원과 사각형을 그린 표시용 영상 반환
img.copy()	메서드	원본을 보존하기 위해 복사본에만 그림
cv2.circle()	함수	모서리에 반지름 25의 원 그리기. -1 은 내부 채우기
cv2.line()	함수	인접한 모서리를 이어 사각형 변 그리기
pt.astype(int)	메서드	실수 좌표를 정수로 변환(그리기 함수는 정수 좌표 요구)
tuple()	함수	넘파이 배열을 (x, y) 튜플로 변환
cv2.LINE_AA	상수	안티에일리어싱 — 원·선의 경계를 부드럽게
cv2.addWeighted(img, 0.3, cpy, 0.7, 0)	함수	원본과 그림을 섞어 반투명 오버레이 효과

실습 코드 (2) — 마우스 콜백

```
53 def onMouse(event, x, y, flags, param):
54     global srcQuad, dragSrc, ptOld, src
55
56     if event == cv2.EVENT_LBUTTONDOWN:
57         for i in range(4):
58             if cv2.norm(srcQuad[i] - (x, y)) < 25:
59                 dragSrc[i] = True
60                 ptOld = (x, y)
61                 break
62
63     if event == cv2.EVENT_LBUTTONUP:
64         for i in range(4):
65             dragSrc[i] = False
66
67     if event == cv2.EVENT_MOUSEMOVE:
68         for i in range(4):
69             if dragSrc[i]:
70                 dx = x - ptOld[0]
71                 dy = y - ptOld[1]
72
73                 srcQuad[i] += (dx, dy)
74
75                 cpy = drawROI(src, srcQuad)
76                 cv2.imshow("img", cpy)
77                 ptOld = (x, y)
78                 break
79
80
81 # 입력 이미지 불러오기
82 src = cv2.imread("./data/scanned.jpg")
83
84 if src is None:
85     print("Image open failed!")
86     sys.exit()
87
88 # 입력 영상 크기 및 출력 영상 크기
89 h, w = src.shape[:2]
90 dw = 500
91 dh = round(dw * 297 / 210) # A4 용지 크기: 210x297cm
92
```

python

```
1 def onMouse(event, x, y, flags, param):
2     global srcQuad, dragSrc, ptOld, src
3
4     if event == cv2.EVENT_LBUTTONDOWN:
5         for i in range(4):
6             if cv2.norm(srcQuad[i] - (x, y)) < 25:      # 모서리 근처를 눌렀는지
7                 dragSrc[i] = True
8                 ptOld = (x, y)
9                 break
10
11     if event == cv2.EVENT_LBUTTONUP:
12         for i in range(4):
13             dragSrc[i] = False
14
15     if event == cv2.EVENT_MOUSEMOVE:
16         for i in range(4):
17             if dragSrc[i]:
18                 dx = x - ptOld[0]
19                 dy = y - ptOld[1]
20
21                 srcQuad[i] += (dx, dy)
22
23                 cpy = drawROI(src, srcQuad)
24                 cv2.imshow("img", cpy)
25                 ptOld = (x, y)
26                 break
```

코드 들여다보기 — 사용한 함수·클래스 (8)

이름	종류	설명
onMouse()	함수	마우스 이벤트 콜백 — (event, x, y, flags, param) 시그니처 고정
global	구문	함수 밖의 srcQuad·dragSrc·ptOld·src를 수정하기 위해 선언
cv2.EVENT_LBUTTONDOWN	상수	왼쪽 버튼 누름 이벤트
cv2.EVENT_LBUTTONUP	상수	왼쪽 버튼 땀 이벤트 → 드래그 상태 해제
cv2.EVENT_MOUSEMOVE	상수	마우스 이동 이벤트 → 드래그 중이면 좌표 갱신
cv2.norm()	함수	두 점 사이 거리 계산 — 25픽셀 이내면 해당 모서리 선택
dragSrc[i]	변수	네 모서리 각각의 드래그 여부를 담은 리스트
srcQuad[i] += (dx, dy)	연산	이동한 만큼 모서리 좌표 갱신

실습 코드 (3) — 메인 루프

```
93 # 모서리 점들의 좌표, 드래그 상태 여부
94 srcQuad = np.array(
95     [[30, 30],
96      [30, h - 30],
97      [w - 30, h - 30],
98      [w - 30, 30]],
99     np.float32
100 )
101 dstQuad = np.array(
102     [[0, 0],
103      [0, dh - 1],
104      [dw - 1, dh - 1],
105      [dw - 1, 0]],
106     np.float32
107 )
108 dragSrc = [False, False, False, False]
109
110 # 모서리점, 사각형 그리기
111 disp = drawROI(src, srcQuad)
112
113 cv2.imshow("img", disp)
114 cv2.setMouseCallback("img", onMouse)
115
116 while True:
117     key = cv2.waitKey()
118     if key == 13: # ENTER 키
119         break
120     elif key == 27: # ESC 키
121         cv2.destroyAllWindows()
122         sys.exit()
123
124 # 투시 변환
125 pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)
126 dst = cv2.warpPerspective(src, pers, (dw, dh), flags=cv2.INTER_CUBIC)
127
128 # 결과 영상 출력
129 cv2.imshow("dst", dst)
130 cv2.waitKey()
131 cv2.destroyAllWindows()
```


python

```
1  # 입력 이미지 불러오기
2  src = cv2.imread("./data/scanned.jpg")
3
4  if src is None:
5      print("Image open failed!")
6      sys.exit()
7
8  # 입력 영상 크기 및 출력 영상 크기
9  h, w = src.shape[:2]
10 dw = 500
11 dh = round(dw * 297 / 210)          # A4 용지 크기: 210x297cm
12
13 # 모서리 점들의 좌표, 드래그 상태 여부
14 srcQuad = np.array([[30, 30], [30, h-30], [w-30, h-30], [w-30, 30]], np.float32)
15 dstQuad = np.array([[0, 0], [0, dh-1], [dw-1, dh-1], [dw-1, 0]], np.float32)
16 dragSrc = [False, False, False, False]
17
18 # 모서리점, 사각형 그리기
19 disp = drawROI(src, srcQuad)
20
21 cv2.imshow("img", disp)
22 cv2.setMouseCallback("img", onMouse)
23
24 while True:
25     key = cv2.waitKey()
26     if key == 13:                # ENTER 키
27         break
28     elif key == 27:              # ESC 키
29         cv2.destroyAllWindows()
30         sys.exit()
31
32 # 투시 변환
33 pers = cv2.getPerspectiveTransform(srcQuad, dstQuad)
34 dst = cv2.warpPerspective(src, pers, (dw, dh), flags=cv2.INTER_CUBIC)
35
36 # 결과 영상 출력
37 cv2.imshow("dst", dst)
38 cv2.waitKey()
39 cv2.destroyAllWindows()
```

코드 들여다보기 — 사용한 함수·클래스 (9)

이름	종류	설명
src.shape[:2]	속성	입력 영상의 (h, w) 추출
round(dw * 297 / 210)	함수	A4 비율(210:297)로 결과 높이 계산

srcQuad / dstQuad	변수	원본 네 모서리와 결과 네 모서리 — 대응 순서 동일
dragSrc	변수	모서리별 드래그 상태 리스트(초기값 모두 False)
cv2.setMouseCallback()	함수	창에 마우스 이벤트 콜백 함수 등록
cv2.waitKey()	함수	키 입력 대기 — 13은 ENTER, 27은 ESC
cv2.destroyWindow()	함수	지정한 이름의 창만 닫기
cv2.getPerspectiveTransform()	함수	확정된 네 모서리로 투시 변환 행렬 계산
cv2.warpPerspective(..., flags=cv2.INTER_CUBIC)	함수	고품질 보간으로 문서를 펴서 출력

조작법: 모서리 원을 **드래그**해 문서에 맞춘 뒤 **ENTER**로 변환, **ESC**로 취소. 좌표 순서만 일관되면 세로 문서·가로 문서 모두 동작함.

7. 정리

변환별 함수 정리표

변환	행렬 생성	적용 함수	핵심 인자
이동	<code>np.array([[1,0,a],[0,1,b]])</code>	<code>warpAffine</code>	a, b (이동량)
전단	<code>np.array([[1,m,0],[0,1,0]])</code>	<code>warpAffine</code>	m (기울기), 출력 크기
크기	— (전용 함수 사용)	<code>resize</code>	dsize 또는 fx·fy, interpolation
대칭	— (전용 함수 사용)	<code>flip</code>	flipCode (+1 / 0 / -1)
1/2 축소	— (전용 함수 사용)	<code>pyrDown</code>	dstsize (생략 시 1/2)
회전	<code>getRotationMatrix2D</code>	<code>warpAffine</code>	center, angle, scale
어파인	<code>getAffineTransform</code>	<code>warpAffine</code>	대응점 3쌍
투시	<code>getPerspectiveTransform</code>	<code>warpPerspective</code>	대응점 4쌍

자주 겪는 함정

상황	증상	해결
----	----	----

행렬 dtype 누락	warpAffine에서 오류 발생	<code>np.float32</code> 명시
dsize를 (0, 0)으로 방치	변환된 부분이 잘림	출력 크기를 직접 계산해 전달
원점 기준 회전	영상이 화면 밖으로 벗어남	<code>getRotationMatrix2D</code> 로 중심 지정
대응점 순서 불일치	결과가 뒤집히거나 꼬임	src·dst 네 점을 같은 순서 로 정렬
확대에 NEAREST 사용	계단 현상 발생	CUBIC·LANCZOS4 사용
축소에 LINEAR 사용	모아레·잡음	<code>INTER_AREA</code> 또는 <code>pyrDown</code>

학습 포인트

- 기하학적 변환 = **좌표를 옮기는 연산** — 화소값 처리(밝기·필터)와 대비해 이해할 것
- 이동·전단·크기·회전은 모두 **2×3 어파인 행렬** 하나로 표현됨
- 동차좌표를 쓰는 이유는 **덧셈(이동)까지 행렬 곱으로 통합**하기 위함
- 임의 축 기준 변환은 **이동 → 변환 → 역이동** 패턴
- 자유도(6 vs 8) = 필요한 **대응점 개수**(3쌍 vs 4쌍)
- 실수 좌표가 생기므로 **보간법 선택**이 품질을 좌우 — 확대는 CUBIC, 축소는 AREA
- 투시 변환의 대표 응용은 **문서·명함 펴기**(OCR 전처리)



PinkLAB YouTube Channel

PinkLAB Studio — Robotics, AI, ROS2 and more!

 **Subscribe on YouTube**

https://www.youtube.com/@pinklab_studio